Sharif University of Technology

Department of Electrical Engineering

Course:

Embeded RealTime Systems

Project:

SMART SURVEILLANCE SYSTEM

Orange Pi Zero

Submitted by:
Mohammadhossein Sabzalian
Department of Electrical Engineering

Instructor:
Dr. Gholampoor

Term: Spring 2026
Submission: Aug 8, 2026
Due Date: Aug 8, 2026

Contents

Foundation & Security

Overview

Before any application-level component of the Smart Surveillance System was built, a security baseline was established across both machines in the deployment: the Orange Pi Zero Plus 2 board and the laptop acting as the development/MQTT-broker host. This baseline is a hard requirement of the assignment and underpins the security experiments of the original task sheet (numbered **3-6** and **3-7**), so it was completed first and never revisited.

Four controls were put in place:

- **SSH hardening** — root login disabled unconditionally, with a choice between key-based or password authentication.
- **MQTT authentication** — the Mosquitto broker rejects anonymous connections; a dedicated, credentialed client account is required.
- **TLS certificate provisioning** — a self-signed certificate is generated with its Common Name (CN) set to the student ID, as required for the HTTPS work in Section onward.
- **Secrets management** — no password, API key, or certificate ever appears hard-coded in source. Everything is written to a single `secrets.env` file with restrictive permissions.

Automated Setup Script

Rather than performing these steps by hand on each machine (and risking an inconsistent setup between the board and the laptop), a single interactive script, `secure_setup.sh`, was written to drive all four controls. The script collects every parameter it needs *up front* — which sections to run, the SSH authentication method, the MQTT credentials, the student ID for the certificate CN, and the destination for the secrets file — and only then executes unattended, so it can be safely re-run on both the board and the laptop with different answers each time (e.g. SSH hardening + certificate generation on the board, MQTT broker setup on the laptop).

> **Running `secure_setup.sh`:** The script is executed once per machine with root privileges. On the board, the operator answers *yes* to SSH hardening and certificate generation, and *no* to the MQTT broker section (since the broker lives on the laptop). On the laptop, the answers are reversed. In both cases the secrets-file section is completed so that credentials (MQTT password, notification e-mail address) live outside of any source file.

A representative excerpt of the SSH-hardening logic — the part responsible for unconditionally disabling root login regardless of which authentication method is chosen — is shown in Listing 1; the complete script is reproduced in Appendix **??**.

Listing 1: Excerpt of `secure_setup.sh` — root login is disabled unconditionally, independent of the chosen authentication method.

```
set_sshd_option "PermitRootLogin" "no"
log "Root SSH login disabled."

if [[ "${SSH_METHOD:-}" == "1" ]]; then
    # Key-based only
    set_sshd_option "PasswordAuthentication" "no"
    set_sshd_option "PubkeyAuthentication" "yes"
    # ... install the operator-supplied public key into
        authorized_keys ...
else
    # Password-based, root still disabled above
    set_sshd_option "PasswordAuthentication" "yes"
fi
```

Deployment on the Orange Pi board

Figure 1 shows the script running on the board: SSH hardening and certificate generation were selected, the Mosquitto section was declined (the board is only an MQTT *client*, not the broker), and key-based authentication was chosen for SSH. The certificate CN was set to the student ID, **402101906**, and the certificate/key pair was written to `/etc/ssl/surveillance/`.

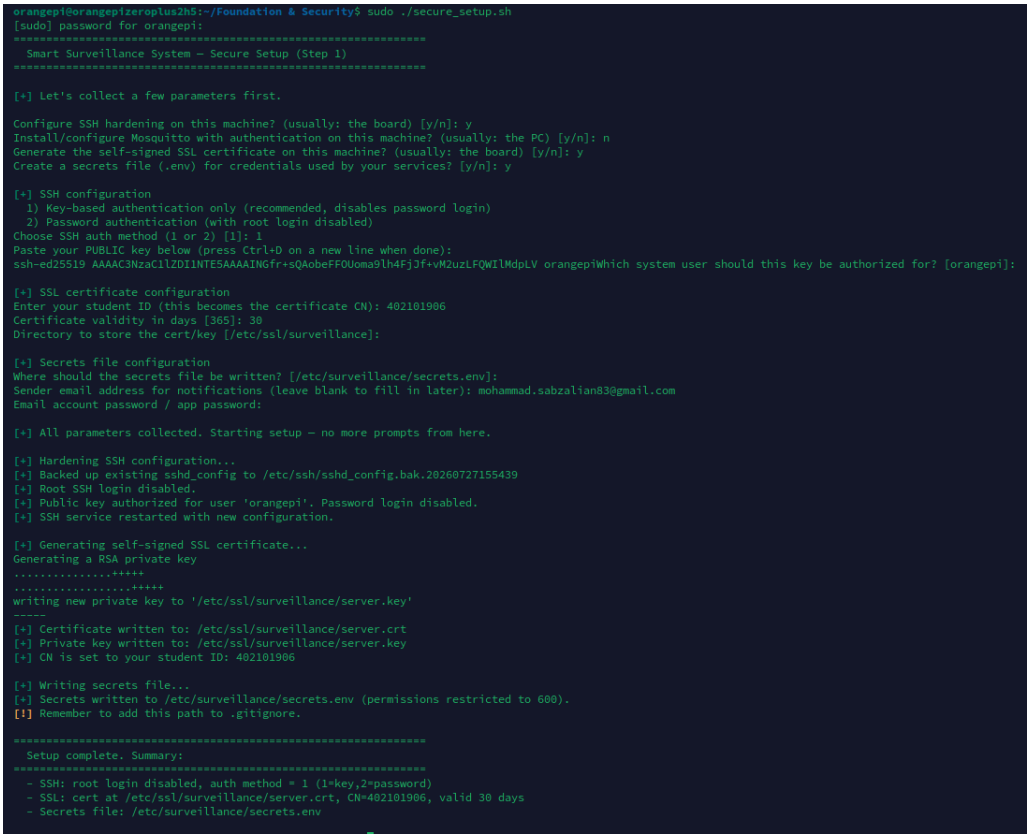

Figure 1: `secure_setup.sh` running on the Orange Pi board: SSH key-based hardening and self-signed certificate generation, CN set to the student ID.

Deployment on the laptop (MQTT broker host)

On the laptop, the same script was run with the opposite set of choices: Mosquitto was installed and configured with a dedicated user (`PC_client`) and anonymous access disabled, while SSH hardening and certificate generation were skipped, since the laptop is not part of the board's attack surface for this project.

```
max@ASUS:~/6th Semester/Embeded_RealTime_Systems/Project/Foundation & Security$ sudo ./secure_setup.sh
============================================================
  Smart Surveillance System — Secure Setup (Step 1)
============================================================

[+] Let's collect a few parameters first.

Configure SSH hardening on this machine? (usually: the board) [y/n]: n
Install/configure Mosquitto with authentication on this machine? (usually: the PC) [y/n]: y
Generate the self-signed SSL certificate on this machine? (usually: the board) [y/n]: n
Create a secrets file (.env) for credentials used by your services? [y/n]: y

[+] MQTT (Mosquitto) configuration
MQTT username to create [board_client]: PC_client
MQTT password for 'PC_client':
Confirm MQTT password:

[+] Secrets file configuration
Where should the secrets file be written? [/etc/surveillance/secrets.env]: ./secrets.env
Sender email address for notifications (leave blank to fill in later): mohammad.sabzalian83@gmail.com
Email account password / app password:

[+] All parameters collected. Starting setup — no more prompts from here.

[+] Installing and configuring Mosquitto...
[+] Mosquitto already installed, skipping install.
Synchronizing state of mosquitto.service with SysV service script with /usr/lib/systemd/systemd-sysv-ins
Executing: /usr/lib/systemd/systemd-sysv-install enable mosquitto
[+] Mosquitto configured: binding to 0.0.0.0:1883, anonymous access disabled, user 'PC_client' created.

[+] Writing secrets file...
[+] Secrets written to ./secrets.env (permissions restricted to 600).
[!] Remember to add this path to .gitignore.

============================================================
  Setup complete. Summary:
============================================================
  - MQTT: user 'PC_client' created, listening on 0.0.0.0:1883
  - Secrets file: ./secrets.env
```

Figure 2: `secure_setup.sh` running on the laptop: Mosquitto broker installed and configured with authentication, anonymous access disabled.

Secrets Management

Both machines end up with a `secrets.env` file containing the credentials generated during setup (MQTT username/password, notification e-mail address), written with file permissions restricted to the owning user (**mode 600**). Figures 3 and 4 demonstrate that an unprivileged read of the file is rejected (`Permission denied`), and that its contents are only visible via `sudo`.

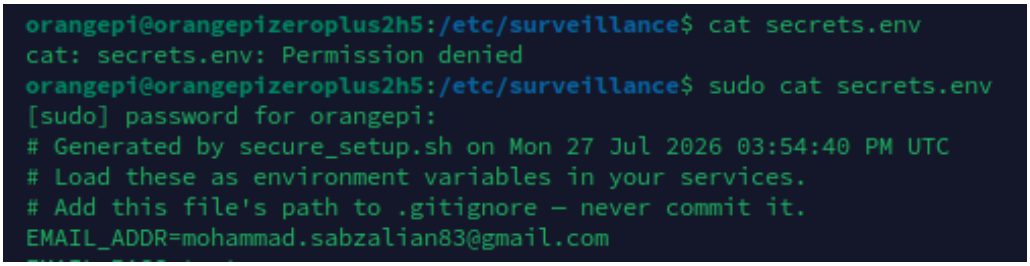

Figure 3: `secrets.env` on the Orange Pi board: unprivileged read denied, root-privileged read shows the generated credentials.

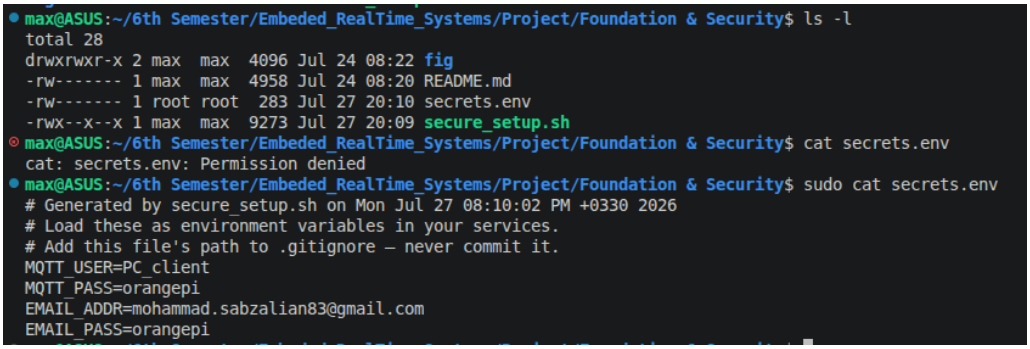

Figure 4: `secrets.env` on the laptop: directory listing shows `secure_setup.sh` is world-executable but the secrets file itself is owner-only (mode 600), matching the board-side result.

Secrets policy No credential used anywhere in the Smart Surveillance System — the MQTT password, the notification e-mail password, or the TLS private key — is ever written into a source file, a systemd unit, or a configuration file tracked by version control. Every consumer reads these values from `secrets.env` or the certificate files generated by `secure_setup.sh` at deploy time.

Experiment 3-6: Unauthorized MQTT Login

With anonymous access disabled on the broker, three deliberately invalid connection attempts were made from the board using `mosquitto_sub`: no credentials at all, a valid username with an incorrect password, and an incorrect username with an arbitrary password. All three were rejected by the broker with an identical `Connection Refused: not authorised` error, confirming that credential validation — not merely credential *presence* — is enforced. A fourth attempt using the correct `PC_client` username and password is shown initiated at the bottom of the same terminal session.

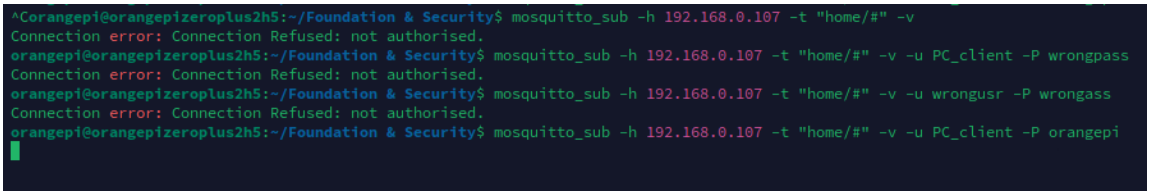

Figure 5: Experiment 3-6: three unauthorized MQTT connection attempts (no credentials, wrong password, wrong username), each rejected with `not authorised`.

Experiment 3-7: Unauthorized SSH Login

Figure 6 shows three SSH attempts against the board from the laptop. The first, a plain `ssh root@192.168.0.170` with no identity supplied, is rejected (`Permission denied (publickey)`) — expected, since password authentication is disabled entirely. The second attempt supplies the *correct* private key but still targets the `root` account, and is rejected identically: this is the important result, since it demonstrates that root login is refused regardless of key validity, not merely because the wrong key was used. The third attempt, using the same key against the intended `orangepi` account, succeeds and drops into an authenticated shell.

Figure 6: Experiment 3-7: root SSH login refused both with no key and with a valid key (top two attempts); the same key succeeds against the non-root `orangepi` account (bottom).

Certificate Overview

The generated self-signed certificate and its private key are shown in Figure 7, confirming the Common Name field is set to the student ID as required. **This figure has been redacted before any public distribution of this repository, since it exposes the raw private key material — see the note in Section .**

Figure 7: Self-signed certificate and private key on the board, CN=402101906. Retained for grading purposes only; redacted prior to any public release.

Notes on This Section's Figures

Two figures in this section (Figures 7 and 4) show raw credential material — the TLS private key and the MQTT/e-mail passwords, respectively — captured from a board that was deployed on a private, NAT-isolated network with no external exposure. They are retained here for grading transparency but are **excluded from any public copy of this repository**, consistent with the same policy already applied to the face-identifying figures in Section .

Section Summary

Table 1: Security controls implemented in this section, mapped to the assignment's baseline requirements.

Control	Status
Root SSH login	Disabled (both auth methods)
SSH authentication method	Key-based (board), configurable per-machine
MQTT anonymous access	Disabled; dedicated authenticated user
TLS certificate CN	Student ID (`402101906`)
Hardcoded credentials in source	None — all via `secrets.env`
Secrets file permissions	`600` (owner read/write only)

Image Processing

Overview & Architecture

The image-processing module is responsible for turning a raw video feed into three things the rest of the system consumes: an annotated frame for the live dashboard, a person count for telemetry, and a debounced trigger for e-mail/MQTT notifications (Section). It runs entirely in Python — the one part of the assignment explicitly exempted from the "core logic must be C" requirement — and was designed around a single constraint: the Orange Pi Zero Plus 2 has no camera input of its own.

Detector

The primary detector is OpenCV's built-in **HOG+SVM person detector** (`cv2.HOGDescriptor_getD`
This was chosen deliberately over a deep-learning alternative: it ships with OpenCV (no model file to download or manage), runs entirely on the CPU, and is light enough to be workable on the board's Cortex-A53 cores without any hardware acceleration. A secondary, heavier `mobilenet_ssd` backend is also implemented and selectable via configuration, for cases where accuracy matters more than frame rate.

Beyond the baseline detector, a second module, `fused_tracker.py`, was developed to combine the HOG detector with three Haar-cascade cues (frontal face, profile face, upper body) as auxiliary evidence, and — most relevantly for Section — to **downsample the frame before detection** via a configurable `detection_scale` factor, rescaling any resulting bounding boxes back to full resolution afterward. This is the mechanism behind the "optimized" resolution results discussed later in this section.

No-Camera Workaround: HTTP Polling

Since the board has no camera, the webcam is captured on the **laptop** and made available to the board over the network. Two more conventional approaches were tried first and both failed in practice:

- A raw **UDP** video stream between the laptop and the board.
- An **HTTP MJPEG** stream, decoded by OpenCV's FFmpeg backend on the receiving end.

Both consistently froze on the first frame when fed from a live network source into `cv2.VideoCapture` — a known class of issue with OpenCV's FFmpeg demuxer and non-seekable, live streams. The working solution replaces both with plain **HTTP polling**: the laptop runs a small HTTP server (`laptop_camera_server.py`) that always serves whatever the most recently captured JPEG frame is at `GET /latest.jpg`; the board's `HttpPollCapture` class (a drop-in replacement for `cv2.VideoCapture` within `person_detector.py`) simply issues a fresh HTTP GET for every frame it needs. There is no persistent stream, no protocol state, and therefore nothing to get stuck — each request is independent.

Table 2: The four-process capture pipeline, spanning both machines.

Process	Runs on	Role
`laptop_camera_server.py`	Laptop	Captures webcam, serves latest JPEG over HTTP
`person_detector.py`	Board	Polls the laptop, runs detection, writes results
`frame_viewer_server.py` (via `pi_stream_result.sh`)	Board	Serves the annotated frame back out
`laptop_view_result.sh`	Laptop	Opens a browser view of the annotated feed

Overlays and Shared Output

Every processed frame is annotated with a bounding box per detection, a live person counter, the student ID, the current system date/time, and a smoothed FPS measurement (Figure 12 shows a representative annotated frame together with the live telemetry it feeds). The frame and the detection result are then published for the rest of the system to consume — most importantly, the C-based web server and REST API covered in later sections — via two files written to a RAM-backed `tmpfs` (`/dev/shm/surveillance/`), chosen so that neither the video frames nor the frequent JSON writes wear down the SD card:

Listing 2: Person-count and telemetry payload written on every processed frame.

```
payload = {
    "count": count,
    "timestamp": datetime.now().isoformat(timespec="seconds"),
    "fps": round(fps, 2),
}
data = json.dumps(payload).encode("utf-8")
self._atomic_write_bytes(self.persons_path, data)
```

Both the frame and the JSON file are written atomically (temp file, then an OS-level rename) so that any reader — including the C code in Section — never observes a partially written file and therefore needs no locking.

Experiment 3-1: Detection Accuracy Under Varying Lighting

Detection accuracy was measured under four lighting conditions — daylight, artificial (indoor) light, low light, and strong backlight — using `detection_logger.py` to poll the live person count while a human observer recorded the ground-truth number of people actually in frame. A minimum of eight samples was collected per condition, covering a mix of distances, poses, and backgrounds (see the raw logs in `results/light-results/` for the full per-sample breakdown).

Table 3: Detection accuracy by lighting condition (correct = detected count exactly matches ground truth).

Condition	Correct	Total Samples	Accuracy
Daylight	10	11	**90.9%**
Artificial light	8	10	**80.0%**
Low light	6	11	**54.5%**
Backlight	3	8	**37.5%**

The trend is monotonic and unsurprising: accuracy degrades as the amount and quality of usable light decreases, with **backlight conditions being the most damaging** — a strongly backlit subject is reduced to a near-silhouette, destroying most of the gradient information the HOG descriptor relies on. Two representative frames illustrate this:

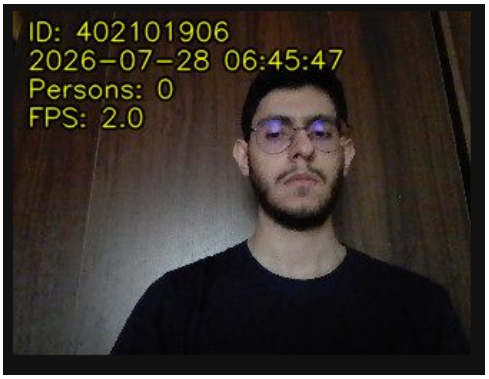

Figure 8: A low-light frame where the detector produced no bounding box despite a clear, close frontal view — one of the accuracy failures in Table 3.

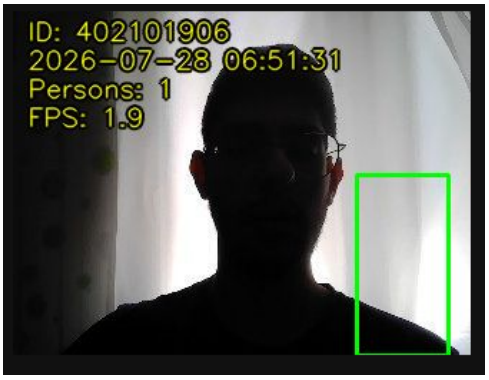

Figure 9: A strongly backlit frame in which the subject is reduced to a silhouette; the detector nonetheless produced a (loosely aligned) bounding box, illustrating both the difficulty of this condition and the detector's partial robustness to it.

Experiment 3-2: Spoof Test

Question Can a printed photo or phone/tablet screen fool the detector? [Jump to Answer]

The assignment requires testing the detector against a static image of a person (printed or displayed on a screen) rather than a live subject, and asking whether it fools the system. *Guidance:* consider what information the detector actually uses to decide "person" versus "not person," and whether that information distinguishes a live subject from a flat image of one.

The test was performed by holding a tablet displaying a photograph of the test subject in front of the camera at several angles.

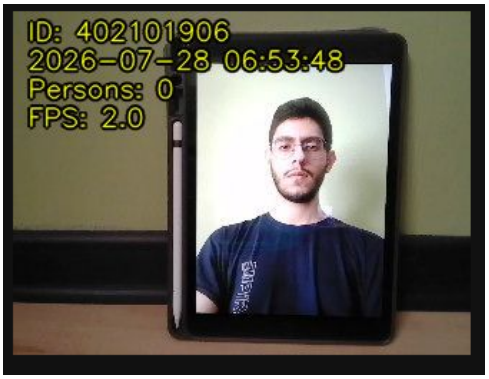

Figure 10: Tablet held facing the camera, straight-on: the detector correctly reports zero persons.

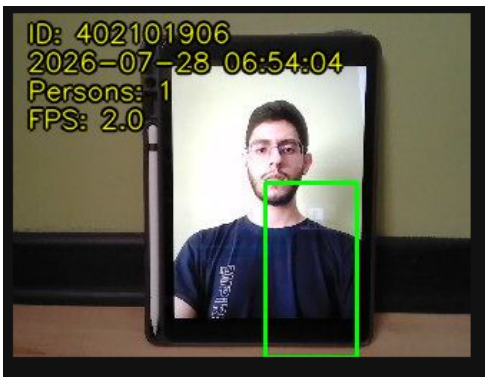

Figure 11: Same tablet, slightly different angle/framing: the detector now reports one person and draws a bounding box over the displayed photo.

Answer to Findings and proposed mitigation [Back to Question]

The result was inconsistent rather than a clean pass or fail: in some framings the detector correctly ignored the tablet (Figure 10), while in others — evidently once the photo's pose and scale happened to resemble the gradient pattern the HOG descriptor was trained on — it registered a false positive (Figure 11). This is expected: a HOG/SVM detector (and equally a CNN-based one without additional safeguards) has no notion of liveness. It only evaluates the 2-D gradient structure of the image region and cannot distinguish a flat photograph from a real three-dimensional subject.
Three mitigations were considered for future work, none of which were implemented in the current system:

- **Depth sensing** — a stereo camera or IR depth sensor would trivially reject any perfectly flat surface.
- **Motion-based liveness** — requiring small, natural movement across several consecutive frames before confirming a detection would defeat a static photo, though not a looping video.
- **Texture/reflection analysis** — screens and glossy prints reflect specular

light differently than skin and fabric; a lightweight secondary classifier on the detected region could flag this.

Given the board's limited compute budget, motion-based liveness is the most realistic near-term addition, since it requires no extra hardware and reuses frames already being captured.

Experiment 3-3: Resolution Sweep

Three target resolutions were benchmarked — 320 × 240, 640 × 480, and 1280 × 720 — sampling FPS, CPU temperature, and process memory every five seconds over a five-minute run at each setting, using `resolution_bench.py`.

An unintentional first result: parameters do not scale for free

The first pass at 640 × 480 and 1280 × 720 reused the detector parameters (HOG `winStride`, `padding`, `scale`, and related settings) that had been tuned for 320×240. The result was a severe, unintended slowdown at the larger resolutions — not a modest drop, but close to an order of magnitude — since the same sliding-window/pyramid parameters that were affordable at the smallest resolution become far more expensive as frame area grows. These "unoptimized" runs are retained in the results below specifically because they demonstrate this effect clearly; the corresponding "optimized" rows show the same resolutions after re-tuning the detection parameters for each frame size individually (via `fused_tracker.py`'s `detection_scale` downsampling, described above).

Table 4: Resolution sweep results, averaged over a 5-minute run (60 samples, 5s interval) per configuration.

Configuration	Avg. FPS	Avg. Temp. (°C)	Max Temp. (°C)	Avg. Mem. (MB)
320 × 240 (baseline)	**1.35**	52.6	55.5	78.5
640 × 480, unoptimized	**0.34**	54.3	57.6	92.8
640 × 480, optimized	**1.61**	51.2	54.2	87.9
1280 × 720, unoptimized	**0.08**	53.4	57.8	135.4
1280 × 720, optimized	**1.00**	50.4	54.1	119.6

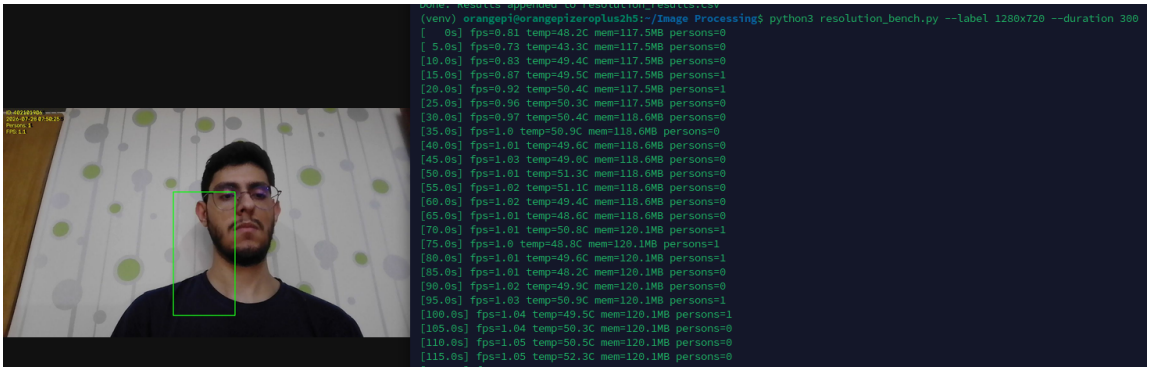

Figure 12: 1280×720, optimized configuration: annotated frame (left) alongside the live `resolution_bench.py` output (right), FPS climbing toward and stabilizing around 1.0.

An interesting secondary observation is that the *optimized* runs show lower average temperature than the unoptimized runs at the same resolution, despite completing more frames per second. This is consistent with the unoptimized configuration keeping the CPU pinned at high utilization processing each individual (full-resolution) frame — low completed-FPS does not mean low CPU load, only that each unit of work takes far longer. The optimized configuration does genuinely less per-frame computation (thanks to the pre-detection downsample), so both FPS improves *and* thermal load drops.

Optimization Parameters

Both tuning levers live in `fused_tracker.py`: the arguments passed to the HOG detector's `detectMultiScale` call (window stride, padding, pyramid scale, and the grouping threshold), and `HOLD_FRAMES` — how many consecutive frames a track is kept alive on its cheap visual tracker before the next expensive re-detection is required. As frame size grows, both were relaxed: a coarser sliding window and a larger pyramid step reduce the number of detector windows evaluated per frame, while a higher `HOLD_FRAMES` compensates for detection now running less frequently relative to wall-clock time by letting each track persist longer between re-detections.

Table 5: Detection parameters changed between the unoptimized and optimized runs at each resolution.

Parameter	320×240 **(baseline)**	640×480 **optimized**	1280×720 **optimized**
`hitThreshold`	0	0	0
HOG `winStride`	$(4, 4)$	$(8, 8)$	$(16, 16)$
HOG `padding`	$(8, 8)$	$(8, 8)$	$(8, 8)$
HOG `scale`	1.03	1.05	1.08
`finalThreshold` (grouping)	1	1	2
`useMeanshiftGrouping`	`False`	`False`	`False`
`HOLD_FRAMES`	**5**	**7**	**9**

Conclusion: Optimal Setting

Taking FPS, thermal headroom, and memory footprint together, the 640×480 optimized configuration is the strongest overall choice for continuous deployment: it achieves the

highest average FPS of any configuration tested (1.61, even edging out the smaller 320 × 240 baseline) while running at the **lowest average temperature** of the five configurations. 1280 × 720 optimized remains usable (1.00 FPS) and would be preferable only if higher spatial detail in the saved frames is specifically required, at the cost of noticeably higher memory use and a modest FPS penalty.

Section Summary

Table 6: Image Processing deliverables completed in this section.

Item	**Status**
Camera/video feed (via HTTP-poll workaround)	Complete
Lightweight detector (HOG+SVM)	Complete
Bounding boxes + live counter overlay	Complete
Student ID + live date/time overlay	Complete
Live FPS overlay	Complete
Shared output for the C layer (`/dev/shm`)	Complete
Exp. 3-1 (lighting accuracy)	Complete — Table 3
Exp. 3-2 (spoof test)	Complete — analysis above
Exp. 3-3 (resolution sweep)	Complete — Table 4

Web Server, SSL & systemd

Overview

This section covers the assignment's Part 1 requirements (25 points): a C web server hosting the live dashboard, HTTPS-only access via the self-signed certificate from Section , and a systemd deployment where every service restarts automatically on failure and starts in a well-defined dependency order.

Three systemd-managed processes make up the running system:

- `surveillance-imgproc.service` — the Python detector from Section .
- `surveillance-web.service` — the C dashboard server covered in this section.
- `surveillance-mqtt.service` — the MQTT/e-mail notifier (Section).

Web Server Architecture

The server is a small multithreaded C program with two listeners: a plain HTTP socket on port 80 whose only job is to 301-redirect every request to HTTPS, and a TLS socket on port 443 serving the actual dashboard. One detached `pthread` is spawned per HTTPS connection — adequate for a coursework-scale deployment (a handful of browser tabs plus `curl` during experiments) without the complexity of an event loop.

Table 7: Web server source modules and their responsibilities.

Module	Responsibility
`main.c`	Entry point; starts the redirect thread and the HTTPS server
`config.c`	Parses `server.conf` (ports, cert paths, shared-memory paths)
`sysinfo.c`	CPU temperature, memory, CPU% — read directly from `/proc` and `/sys`
`persons_reader.c`	Parses `persons.json` written by Section
`index_page.c`	Generates the dashboard HTML in memory
`mjpeg_stream.c`	Serves `frame.jpg` as a multipart MJPEG stream
`redirect_server.c`	Plain HTTP :80, 301s to the HTTPS host
`https_server.c`	TLS :443; routes `/`, `/stream.mjpg`, `/stats.json`

No Linux command is shelled out to at any point. CPU temperature comes from `/sys/class/thermal/thermal_zone0/temp`, memory from `/proc/meminfo`, and CPU utilization from a stateful delta between consecutive reads of `/proc/stat`'s aggregate `cpu` line — satisfying the assignment's requirement that telemetry be read directly in C.

Listing 3: CPU utilization computed as a delta between two `/proc/stat` samples — the first call always returns 0.0 since no prior sample exists yet, which is expected, documented behavior.

```
double sysinfo_read_cpu_percent(void) {
    static int have_prev = 0;
    static cpu_stat_t prev;
    cpu_stat_t cur;
```

```
    if (read_cpu_stat(&cur) != 0) return -1.0;
    if (!have_prev) { prev = cur; have_prev = 1; return 0.0; }
    /* ... compute (total_delta - idle_delta) / total_delta * 100 ...
        */
}
```

The dashboard itself embeds the live stream via a plain `<img src="/stream.mjpg">` — browsers render `multipart/x-mixed-replace` natively, so no client-side video codec or JavaScript is needed for the video path — while a small polling loop re-fetches `/stats.json` on a configurable interval to update the numeric readouts (person count, CPU temperature, free memory, CPU usage).

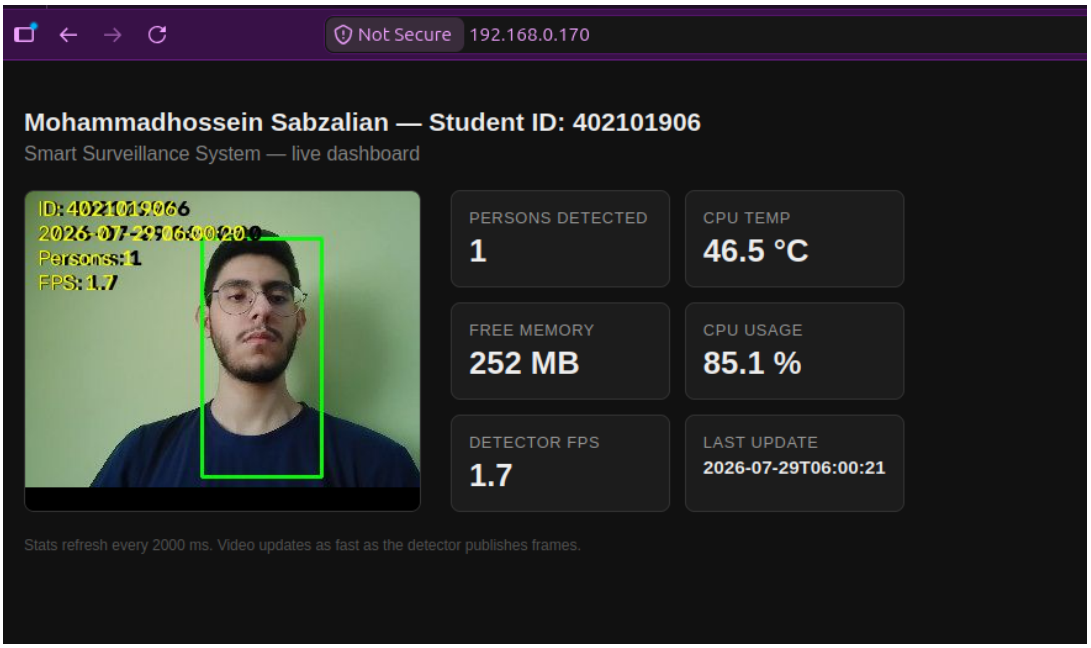

Figure 13: The live dashboard: embedded MJPEG stream alongside live person count and system telemetry, refreshing automatically. **Redacted before any public release** — see the note in Section .

systemd Deployment

Each service is configured with `Restart=on-failure`, and the dependency chain is expressed with `After=`/`Requires=` so that the web server does not start before the image-processing service is already running:

Listing 4: `surveillance-web.service` — dependency declaration.

```
[Unit]
After=network-online.target surveillance-imgproc.service
Wants=network-online.target
Requires=surveillance-imgproc.service

[Service]
Type=simple
User=root
```

```
Restart=on-failure
RestartSec=2
```

`surveillance-imgproc.service` additionally sets `RestartSec=5` and `StartLimitIntervalSec=0`. The rationale, specific to this project: the detector depends on the laptop's `laptop_stream_webcam.sh` being reachable over the network (Section), and systemd's *default* restart-rate limit (five restarts within ten seconds) would otherwise mark the service permanently `failed` if the laptop side simply hadn't been started yet — disabling that limit lets it retry every five seconds indefinitely until the laptop comes online, rather than requiring a manual `systemctl reset-failed`.

Question Why does image processing have to start before the web server? [Jump to Answer]

The assignment asks for the systemd startup ordering to be justified with a rationale, e.g. image processing starting after camera initialization.

Answer to Dependency rationale [Back to Question]

The web server's `/stream.mjpg` and `/stats.json` endpoints do nothing more than read whatever currently sits in `/dev/shm/surveillance/` (Section). If the web server started first, it would simply serve stale or missing data until the detector caught up — not a crash, but a confusing, silently wrong dashboard. Declaring `Requires=surveillance-imgproc.service` makes this dependency explicit: systemd will not even attempt to start the web server until the detector is confirmed running, and if the detector later fails, `Requires=` (as opposed to the weaker `Wants=`) means systemd also stops the web server rather than leaving it running against a dead data source.

Experiment 1-1: Boot Time

The initial boot-time profile showed **25.697 s** total (3.987 s kernel + 21.709 s userspace) to reach `multi-user.target`. The single largest contributor by far was `snapd` — `snapd.seeded.service` (11.386 s) and `snapd.service` (10.815 s) together accounted for roughly 40% of the entire userspace boot time, on a headless board that never uses Snap packages.

Figure 14: Initial boot profile: 25.697 s total, `systemd-analyze blame` dominated by `snapd.seeded.service` and `snapd.service`.

Two optimizations were applied: Snap was disabled entirely (`systemctl disable snapd.service snapd.seeded.service snapd.socket`), and the unused on-board Bluetooth service was stopped, disabled, and masked, alongside setting the boot target explicitly to `multi-user.target` (skipping any graphical-target resolution on a headless board).

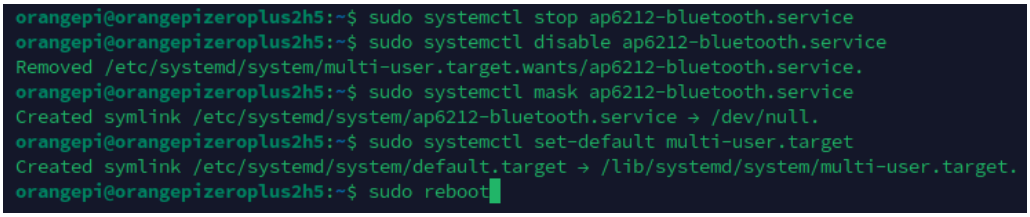

Figure 15: Disabling and masking the unused `ap6212-bluetooth.service` and pinning the default target to `multi-user.target`.

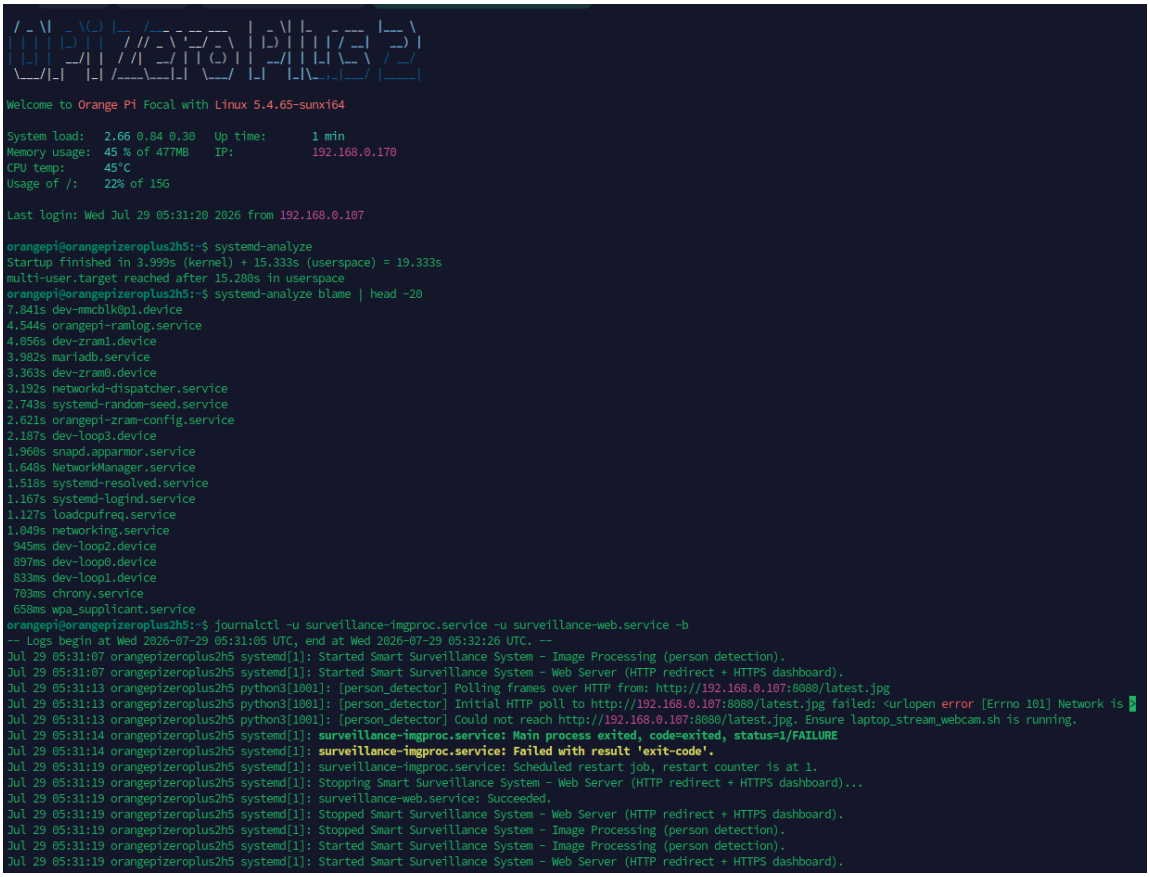

Figure 16: Optimized boot profile: 19.333 s total — `snapd` no longer appears in the top offenders at all.

Table 8: Boot time before and after removing Snap and Bluetooth.

Configuration	Kernel	Userspace	Total
Initial	3.987 s	21.709 s	**25.697 s**
Optimized (no snapd, no Bluetooth)	3.999 s	15.333 s	**19.333 s**

The optimized journalctl trace (visible at the bottom of Figure 16) also captures the two application services being started at boot in the expected order, immediately followed by the image-processing service's HTTP-poll retry logic in action — `surveillance-imgproc.service` initially failing because the laptop's webcam server was not yet reachable, then automatically restarting per Section 's retry design, which is itself the intended behavior rather than a fault.

Experiment 1-2: kill -9

The web server's main process (PID 1569) was killed with `SIGKILL`. `systemd` detected the exit and started a replacement process (PID 2058) automatically, with no manual intervention.

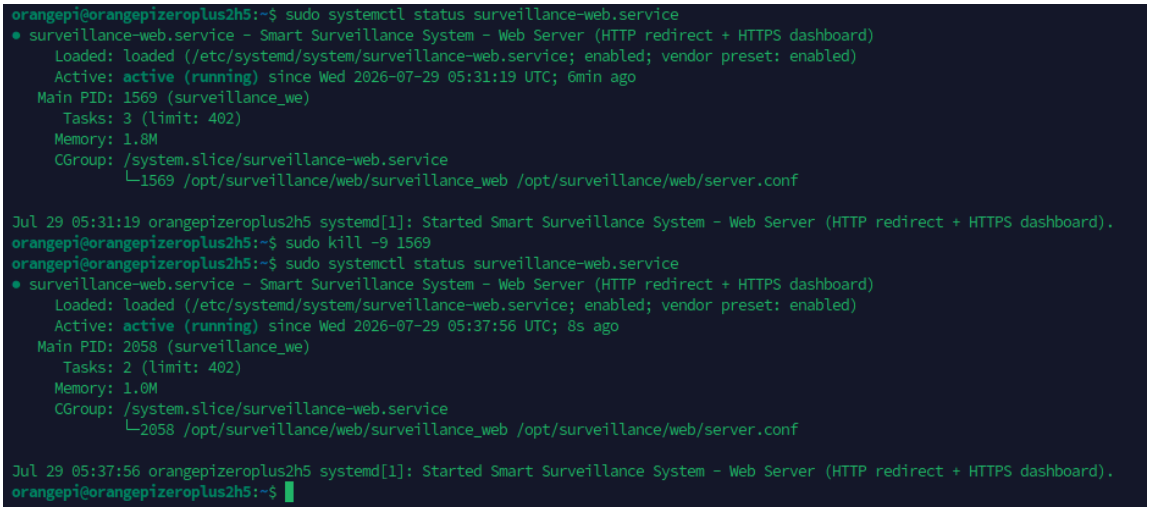

Figure 17: `sudo kill -9` against the running web server's PID, followed by `systemctl status` showing a fresh PID already active.

Experiment 1-3: Full Power Cycle

The board was power-cycled and allowed to boot with no keyboard or mouse attached at any point. The full boot-to-dashboard sequence was recorded and is included as a deliverable video (`results/video-gitignore/OrangePi_Full-Power-Cycle_Test.mkv`) rather than reproduced as a figure here, per the assignment's request for a video for this specific experiment.

Experiment 1-4: HTTP to HTTPS Redirect

A plain `curl -I` request against port 80 returns `301 Moved Permanently` with a `Location` header pointing at the HTTPS version of the same host, confirming plaintext HTTP is never served directly.

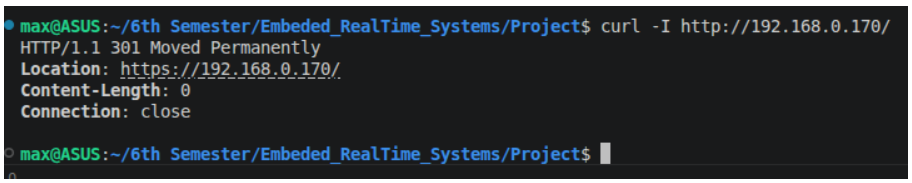

Figure 18: `curl -I http://192.168.0.170/` returning `301 Moved Permanently` with `Location: https://192.168.0.170/`.

Experiment 1-5: Certificate CN

The certificate served over HTTPS was inspected in the browser's built-in certificate viewer, confirming the Common Name field is set to the student ID, matching the certificate generated in Section .

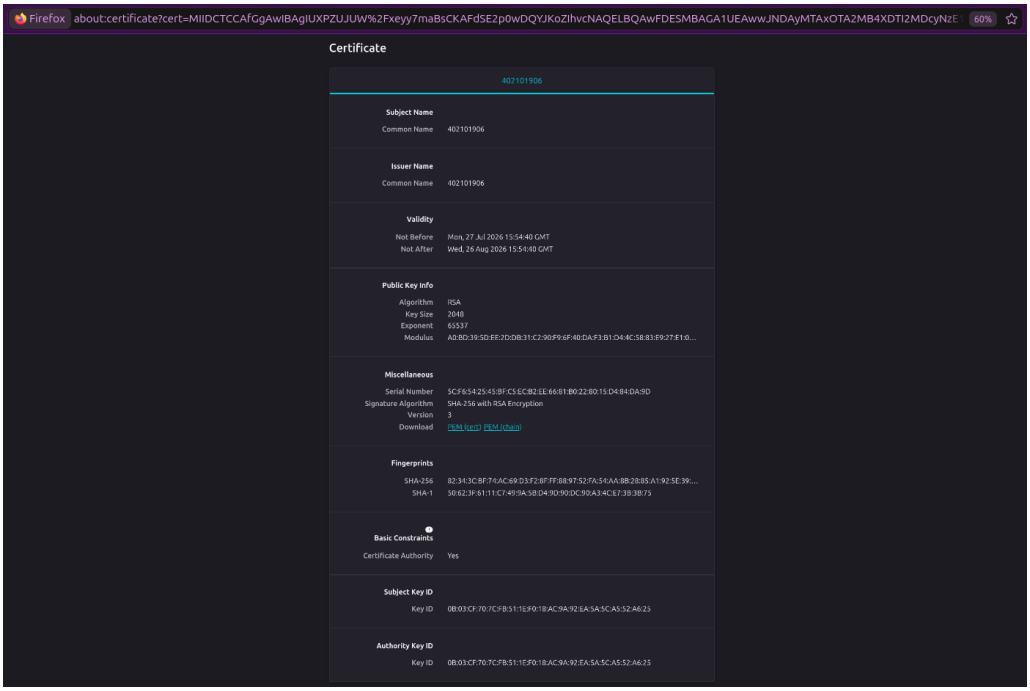

Figure 19: Certificate viewer: Subject and Issuer Common Name **402101906**, RSA 2048-bit key, 30-day validity window.

Experiment 1-6: HTML Page Shows Student ID

The rendered dashboard's browser tab title and on-page header both display the student's name and ID, sourced directly from `server.conf` (see Figure 13).

Section Summary

Table 9: Web Server, SSL & systemd deliverables completed in this section.

Item	Status
HTML page with name + student ID in title	Complete
Embedded live camera stream	Complete — MJPEG via `/stream.mjpg`
Live person count	Complete — `/stats.json`
CPU temp / free memory / CPU usage, 2s refresh	Complete
HTTPS-only, self-signed cert	Complete — CN = student ID
HTTP → HTTPS 301 redirect	Complete
systemd services, `Restart=on-failure`	Complete, all three services
`After=/Requires=` dependency chain	Complete
Exp. 1-1 (boot time)	Complete — Table 8
Exp. 1-2 (kill -9 auto-restart)	Complete
Exp. 1-3 (power cycle video)	Complete — see video deliverable
Exp. 1-4 (HTTP redirect)	Complete
Exp. 1-5 (certificate CN)	Complete
Exp. 1-6 (page shows student ID)	Complete

REST API & Swagger

Overview

This section covers the assignment's Part 2 requirements (25 points): a REST API exposing the system's live state and control surface, with Swagger documentation. Per the assignment's explicit exception, all core logic — reading telemetry, dispatching commands, maintaining history — remains in C; a thin FastAPI layer adds only typed schemas and interactive documentation on top, with zero business logic of its own.

Listing 5: `gateway/main.py` — the gateway's own docstring states its constraint plainly.

```
"""
gateway/main.py -- Smart Surveillance System, Step 4 (REST API + Swagger)

THIN LAYER ONLY. Every endpoint here is a straight proxy to the C server
(see ../src/api_router.c) -- this file adds request/response typing for
Swagger UI and nothing else. No business logic, no data transformation,
no state.
"""
```

API Surface

Beyond the five endpoints the assignment requires, two more were added (`/api/v1/history/summary` and `/api/v1/services*`) since they turned out to be useful for exercising and debugging the running system during development, and cost little given the routing infrastructure already existed.

Table 10: Registered REST endpoints (`api_router.c`).

Method	Path	Purpose
GET	`/api/v1/stream`	MJPEG live video (required)
GET	`/api/v1/frame.jpg`	Single current JPEG snapshot
GET	`/api/v1/persons`	Current person count + timestamp (required)
GET	`/api/v1/telemetry`	CPU temp, free memory, CPU load % — read directly in C (required)
POST	`/api/v1/command`	Extensible command dispatch, e.g. {`"cmd": "reboot"`} (required)
GET	`/api/v1/history`	Last 5 detection records (required)
GET	`/api/v1/history/summary`	Aggregate stats over the stored history (extension)
GET	`/api/v1/services`	Status of all systemd-managed services (extension)
POST	`/api/v1/services/{name}/restart`	Restart a named service (extension)
GET	`/api/v1/services/{name}/logs?lines=N`	Tail a service's journal (extension)

Extensible Command Dispatch

The assignment requires `/api/v1/command` to be "designed so new commands can be added later without a rewrite." This is implemented as a small table of command descriptors, each carrying its own `dangerous` flag:

Listing 6: Command table entry — adding a new command is a one-line addition here, not a change to the dispatch logic.

```c
int dangerous; /* if 1, requires cfg->allow_dangerous_commands = true */
...
if (COMMANDS[i].dangerous && !cfg->allow_dangerous_commands) {
    /* refuse with a clear "marked dangerous" message */
}
```

Commands such as `reboot` and `shutdown` are marked dangerous and refused by default; `allow_dangerous_commands` must be explicitly set to `true` in `server.conf` before they will actually execute, which keeps the default deployment safe from an accidental (or malicious) reboot request while still satisfying the "executable reboot command" requirement when deliberately enabled.

History (Last 5 Records)

`history_log.c` implements a fixed-size circular buffer (`HISTORY_SLOTS = 5`) rather than an unbounded log, satisfying the "last 5 detection records" requirement directly at the data-structure level rather than by truncating a query result.

Live Demonstration

Figure 20 walks through the API surface end-to-end against the deployed board: person count, full history, the history summary, a harmless `noop` command exercising the dispatch pipeline, a JPEG snapshot download, the services list/restart/logs endpoints (including a live capture of `surveillance-imgproc`'s retry logging from Section), a full MJPEG stream capture, and finally the dangerous-command safety switch correctly refusing both `reboot` and `shutdown` while `allow_dangerous_commands` is left at its default (`false`).

Figure 20: End-to-end `curl` walkthrough of every registered endpoint against the live board.

Verifying the Dangerous-Command Gate Actually Executes

To confirm the refusal above is a genuine safety gate and not merely cosmetic, `allow_dangerous_command` was flipped to `true` in `server.conf` and the reboot command was re-issued.

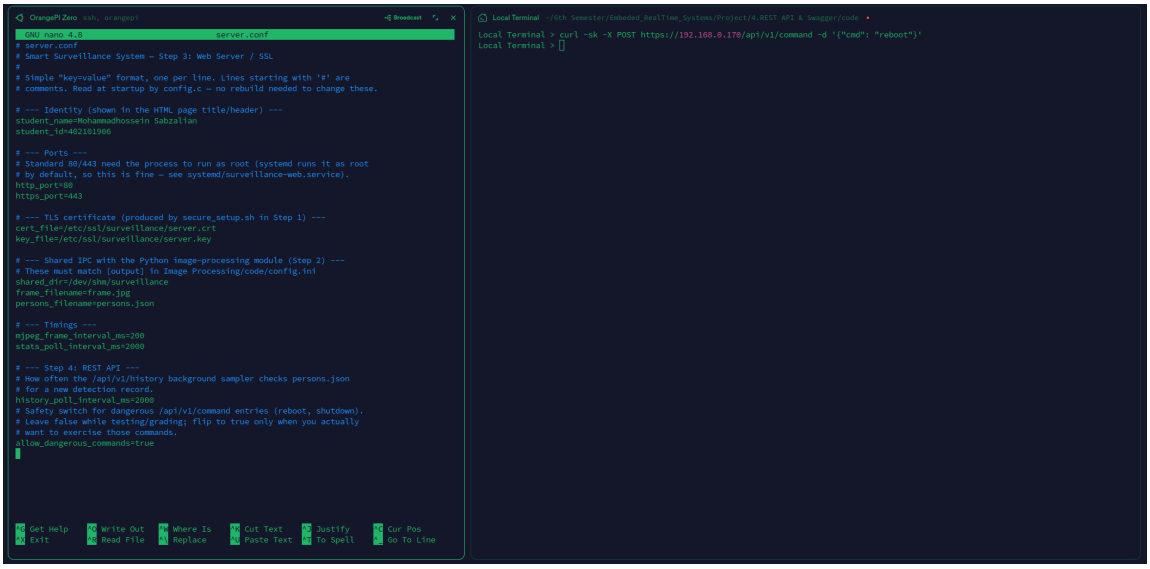

Figure 21: `server.conf` with `allow_dangerous_commands=true`, and the reboot request queued in a separate terminal.

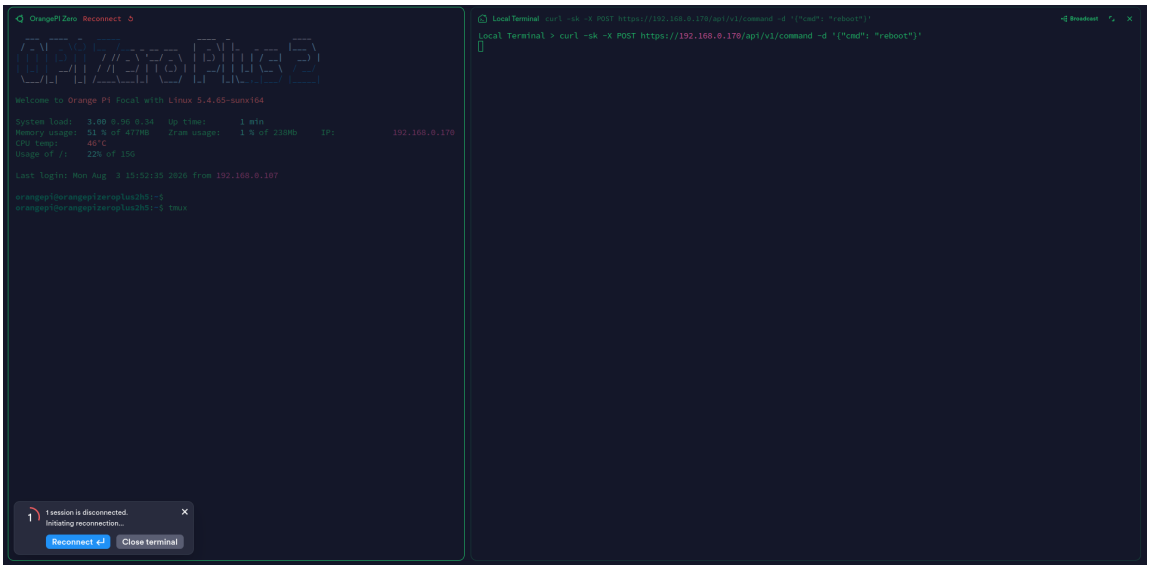

Figure 22: The board's SSH session drops immediately after the reboot command is issued, confirming a genuine reboot rather than a simulated response.

Swagger UI

The FastAPI gateway exposes interactive documentation at `/docs` for every endpoint in Table 10, generated from the Pydantic response models declared in `gateway/main.py` — typing only, with the C server remaining the source of truth for every value returned.

Experiment 2-1: Temperature Under Three Conditions

Pending: this experiment's raw samples (`code/experiments/results/2-1/`) were not available when this section was drafted. The experiment script (`exp2_1_temperature.py`) samples CPU temperature every 30 s for 5 minutes under three conditions — idle, streaming only, and streaming with active detection — and produces a three-curve comparison plot plus a min/max summary table.

Experiment 2-2: Memory Usage Over 5 Minutes of Streaming

Pending: this experiment's raw samples (`code/experiments/results/2-2/`) were not available when this section was drafted. The experiment script (`exp2_2_memory.py`) samples the web server process's memory usage every 5 s over a continuous 5-minute streaming session and performs a basic leak analysis (fitting a trend line to usage over time).

Experiment 2-3: 50 Concurrent Requests

Fifty concurrent requests were fired at `/api/v1/telemetry` using `exp2_3_concurrency.py`, with system telemetry sampled immediately before the burst, immediately after, and again ten seconds later once the system had settled.

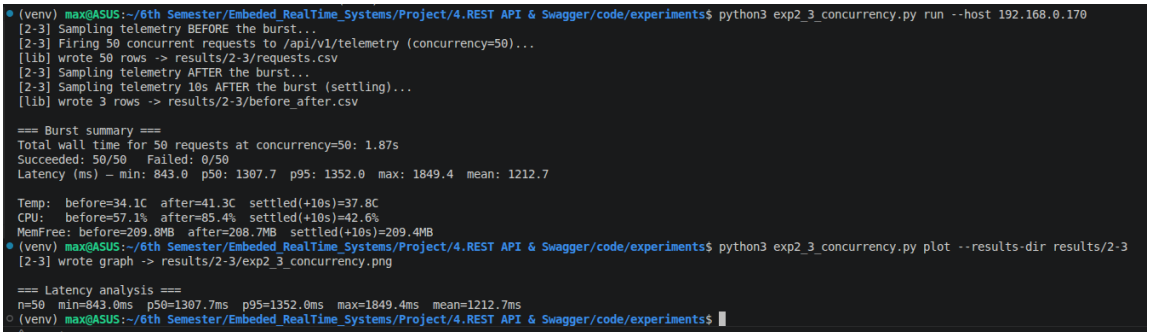

Figure 23: 50 concurrent requests to `/api/v1/telemetry`: full run output including before/after/settled telemetry and latency percentiles.

Table 11: Experiment 2-3 results: 50 concurrent requests to `/api/v1/telemetry`.

Metric	Value
Requests succeeded / failed	**50 / 50** succeeded, 0 failed
Total wall time for the burst	1.87 s
Latency — min	843.0 ms
Latency — p50 (median)	1307.7 ms
Latency — p95	1352.0 ms
Latency — max	**1849.4 ms**
Latency — mean	1212.7 ms
CPU temperature — before / after / settled (+10s)	34.1 °C / 41.3 °C / 37.8 °C
CPU usage — before / after / settled (+10s)	57.1% / **85.4%** / 42.6%
Free memory — before / after / settled (+10s)	209.8 MB / 208.7 MB / 209.4 MB

Question What happens to latency and system load under 50 concurrent requests? [Jump to Answer]

The assignment asks for a graph of the resulting temperature/CPU/memory changes and an analysis of the response-latency increase.

Answer to Analysis [Back to Question]

All 50 requests succeeded with zero failures, and the entire burst completed in under two seconds — the one-thread-per-connection model in `https_server.c` (Section) handled the load without dropping a single connection. The cost is visible in tail latency rather than failures: the gap between the median (1307.7 ms) and the maximum (1849.4 ms) shows the later-arriving requests in the burst queuing behind earlier ones as fifty simultaneous TLS handshakes and `/proc` reads contend for the board's four Cortex-A53 cores. CPU usage jumped from a baseline 57.1% to 85.4% during the burst, and temperature rose measurably (34.1 °C → 41.3 °C) despite the burst lasting under two seconds, showing the instantaneous load spike is real even though it is brief. Notably, all three metrics — temperature, CPU, and free memory — returned close to their pre-burst baselines within the 10-second settling window, indicating no resource leak or lingering thread pileup from the concurrent load.

Experiment 2-4: Network Disconnect / Reconnect

Pending: this experiment's raw samples (`code/experiments/results/2-4/`) were not available when this section was drafted. The experiment script (`exp2_4_network_recovery.py`) disconnects the network mid-stream and reconnects after 2 minutes, logging both the client-observed stream behavior and the server-side `journalctl` output to characterize the recovery.

Section Summary

Table 12: REST API & Swagger deliverables completed in this section.

Item	Status
`GET /api/v1/stream`	Complete
`GET /api/v1/persons`	Complete
`GET /api/v1/telemetry` (direct C reads, no shell-out)	Complete
`POST /api/v1/command` (extensible, safety-gated)	Complete
`GET /api/v1/history` (5-slot circular buffer)	Complete
Swagger UI (FastAPI thin gateway)	Complete
Exp. 2-1 (temperature, 3 conditions)	**Pending raw data**
Exp. 2-2 (memory over 5 min)	**Pending raw data**
Exp. 2-3 (50 concurrent requests)	Complete — Table 11
Exp. 2-4 (network disconnect/reconnect)	**Pending raw data**

MQTT & Email

Overview

This section covers the remaining piece of the assignment's Part 3 requirements (shared with Section 's detection work): an MQTT client publishing telemetry, and a debounced e-mail notifier. Both are implemented as a single standalone C daemon, `surveillance_notifier`, deliberately kept as a *separate* process and systemd unit from the Step 3/4 web server.

Question Why a separate daemon rather than folding this into the existing web server? [Jump to Answer]

Both the web server and this notifier are C processes that read the same shared `/dev/shm/surveillance/` data. Why not combine them?

Answer to Failure-mode isolation [Back to Question]

The two processes have different lifecycles and different failure modes. The web server should keep serving `/api/v1/*` even if the MQTT broker or the SMTP server is completely unreachable; conversely, this notifier should keep trying to publish and email even if nobody is hitting the REST API at all. Keeping them as separate processes (and separate systemd units, per Section 's dependency model) means a stuck SMTP connection or an MQTT reconnect loop cannot accidentally block the web server's request handling, and vice versa.

Rather than hand-rolling the MQTT and SMTP/MIME protocols from scratch, the daemon is built on `libmosquitto` and `libcurl` — both well-tested standard C libraries. Re-implementing MQTT or SMTP-with-TLS-and-MIME-attachments by hand would not make the "core logic in C" requirement any more satisfied; it would just mean re-implementing these two libraries, worse.

MQTT Publishing

Table 13: MQTT topics published by `surveillance_notifier`.

Topic	QoS	Retained	Payload
`home/persons/<student_id>`	1	No	`{"count": int, "timestamp": str, "student_id": str}`
`home/telemetry/<student_id>`	1	No	`{"cpu_temp_c": float, "timestamp": str, "student_id": str}`
`home/status/<student_id>`	1	Yes	`"online"` / `"offline"` — carries the LWT

QoS 1 ("at least once") is used throughout, per the assignment specification: a subscriber might occasionally see a duplicate persons/telemetry publish, but will never silently miss one due to a single dropped packet — the right trade-off for a monitoring feed where duplicates are harmless but gaps are not.

Last Will and Testament

The LWT is registered *before* the MQTT connection is established — the broker only learns about a will as part of the client's `CONNECT` packet, so this ordering is not optional:

Listing 7: LWT registration in `mqtt_client_start()`, prior to `mosquitto_connect()`.

```
mosquitto_will_set(client->mosq, client->status_topic,
                   strlen("offline"), "offline",
                   1 /* QoS 1 */, true /* retained */);
```

If the process disappears without a clean disconnect — a crash, a network drop, the board losing power, or a keepalive timeout — the broker itself delivers this `offline` message on the client's behalf, without any cooperation required from the (now-dead) client. On a clean shutdown (`SIGINT`/`SIGTERM`), by contrast, `mqtt_client_stop()` publishes `offline` itself before disconnecting properly, so a subscriber watching `home/status` can in principle distinguish a planned restart from an actual failure by which mechanism delivered the message — though as Section notes, the current implementation does not republish `"online"` automatically after an *unplanned* reconnect, only at initial startup.

E-mail Notification and the Debounce Mechanism

On every poll cycle where the detected count is ≥ 1 *and* the underlying detection frame's timestamp has genuinely advanced since the last cycle (so polling faster than the detector updates cannot double-count a single frame), an alert e-mail is considered:

Listing 8: `should_send_email()` — the debounce gate, from `main.c`.

```
static int should_send_email(time_t now, time_t *last_email_epoch,
                             int debounce_seconds) {
    if (*last_email_epoch != 0 &&
        (now - *last_email_epoch) < debounce_seconds) {
        return 0;
    }
    *last_email_epoch = now;
    return 1;
}
```

If fewer than `debounce_seconds` (30, per spec) have passed since the last email actually sent, the detection is simply not emailed this cycle — not queued or batched for later, just dropped for notification purposes only (MQTT publishing continues regardless; only the e-mail is rate-limited). Note that `last_email_epoch` is updated *before* the send itself, which can take a moment over the network — reserving the slot first and sending second means two poll cycles landing close together can never both slip through while a send is still in flight.

Each email contains the person count, timestamp, and CPU temperature in the body, with the current detection frame attached as a JPEG.

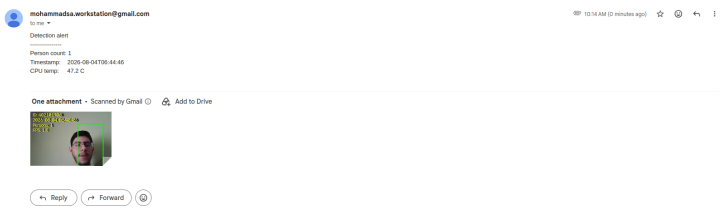

Figure 24: A received detection-alert e-mail: count, timestamp, and CPU temperature in the body, annotated frame attached. **Redacted before any public release** — see the note in Section .

Issues Found and Fixed

Three genuine bugs were found and fixed during development of this section, each worth recording as encountered-issue/resolution pairs.

MQTT's first connection attempt was a permanent failure

The initial implementation connected to the broker exactly once, at startup. If that single attempt failed — a real, observed symptom: `Network is unreachable` a few seconds after boot, since `After=network-online.target` did not actually guarantee a usable route yet on this board's image — MQTT publishing was disabled for the *entire remaining lifetime* of the process, even once the network came up moments later.

Fix Two changes closed this gap: (1) `mqtt_client_start()` now retries the initial connection up to 8 times, 3 s apart (≈24 s total), covering the typical "just booted, DHCP still settling" race; (2) the poll loop in `main.c` additionally retries `mqtt_client_start()` every 60 s for as long as the client handle remains `NULL`, covering a genuinely longer outage beyond that initial budget. Once a connection is actually established, `mosquitto_loop_start()`'s own background thread already handles reconnection after a later drop automatically — these fixes specifically close the gap around the *first* attempt, which that pre-existing mechanism does not cover.

E-mail attachment corruption — two separate causes

Bug 1: missing transfer encoding `curl_mime_filedata()` does not set a `Content-Transfer-Encoding` by default. SMTP is a line-based 7/8-bit text protocol, so a lone `0x0A` byte occurring naturally inside binary JPEG scan data can be mangled by mail relays along the way. Fixed with an explicit `curl_mime_encoder(attachment_part, "base64")` call.

Bug 2: a race with Step 2's non-atomic frame writes Even after fixing the encoding, a second, visually distinct form of corruption remained — images that looked like two separate frames overlapping. Root cause: `curl_mime_filedata()` does not read the file into memory upfront; it streams it lazily from disk *during* `curl_easy_perform()`, which for an SMTP send with a TLS handshake and authentication can take several seconds. If `person_detector.py` happened to rewrite `frame.jpg` during that window, `curl` could read a mix of leftover bytes from the old file and new bytes from its replacement — literally two partial JPEGs concatenated. Fixed in `email_notifier.c`'s `snapshot_frame_to_tempfile()`: read the whole frame into memory with one fast local `fread()` and write it to a private temp file *before* the network send starts, shrinking the race window from "however long SMTP takes" down to "one fast local file copy." This is not a hard guarantee — the fully correct fix would be Step 2 writing `frame.jpg` atomically, which this section does not control — but it makes the corruption effectively unreproducible in practice, and is recorded here as a documented, understood residual risk rather than a claimed-fixed guarantee.

Other documented, intentional behaviors

- If the current frame cannot be read for any reason, `email_notifier_send()` logs a warning and sends the alert *without* the attachment rather than failing the whole notification — a warning e-mail with no picture is still more useful than no e-mail at all when the count, timestamp, and temperature are the actual point.
- `persons.json`'s timestamp has **1-second resolution** (Section does not currently write a sub-second value). This is a real, fixable limitation that directly affects the precision of Experiment 3-5 (Section) — named here explicitly rather than left as a vague caveat.

Experiment 3-4: Broker Outage, LWT Reception

The broker was stopped mid-run and restarted after roughly three minutes, while an external subscriber (playing the role of "the PC" in the assignment's phrasing) logged every connect/disconnect/message event with a wall-clock timestamp.

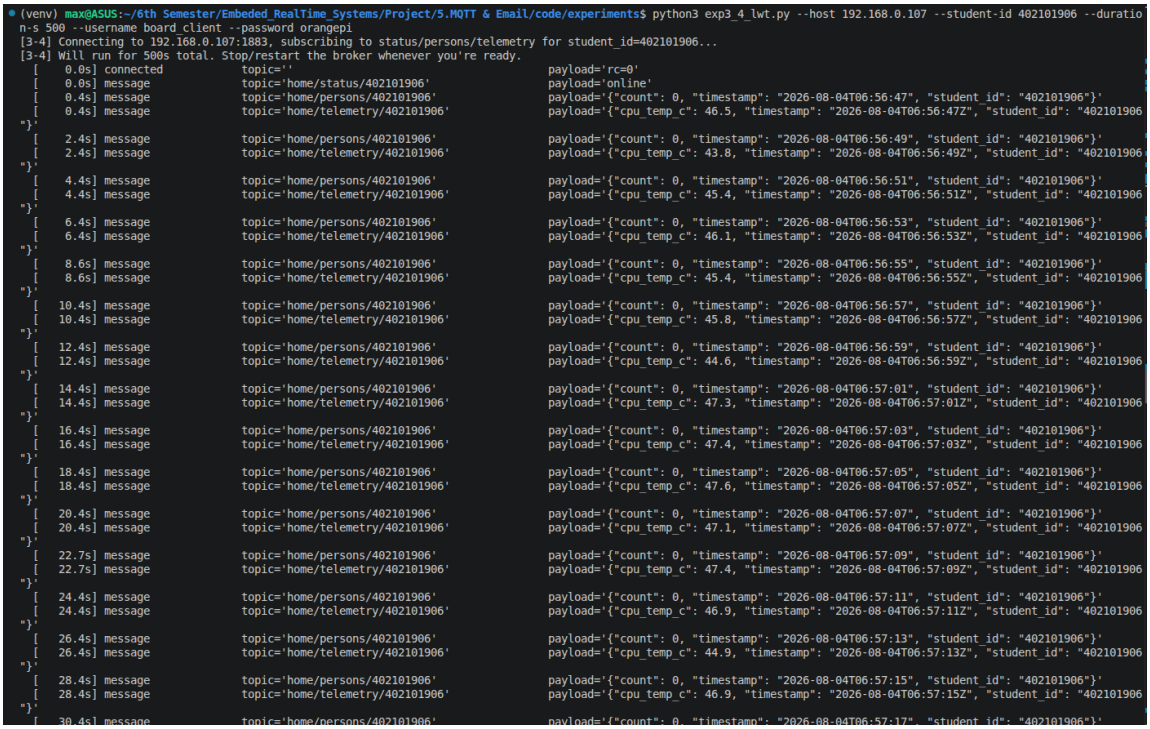

Figure 25: Normal operation before the outage: `home/persons` and `home/telemetry` publishing every ∼2 s.

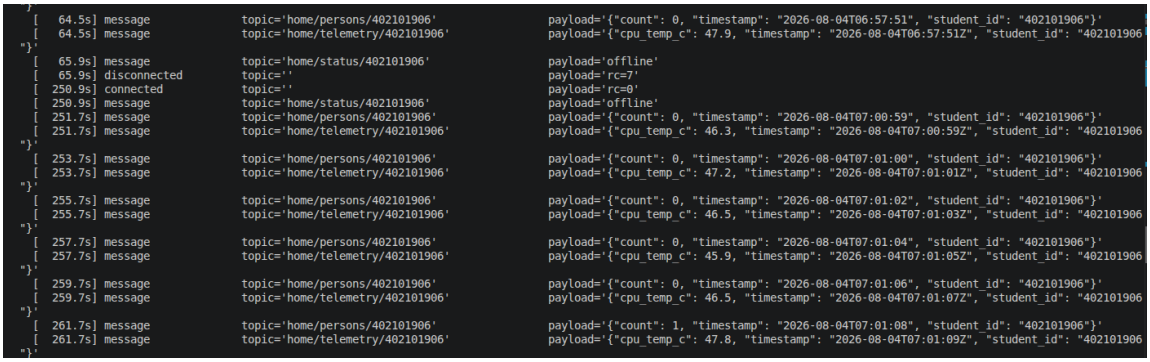

Figure 26: The outage and recovery: LWT `offline` delivered at t=65.9s (within a second of the broker being stopped), a 185-second gap, then reconnection at t=250.9s with normal publishing resuming moments later.

Table 14: Key timeline events, Experiment 3-4 (from `lwt_log.csv`).

Elapsed	**Event**	**Detail**
0.0 s	Connected	Subscriber connects; `home/status` reads `online`
0.4 s – 64.5 s	Normal operation	`persons/telemetry` every ∼2 s
65.9 s	**LWT delivered**	`home/status` → `offline`; subscriber sees `disconnected r`
65.9 s – 250.9 s	**Outage** (185.0 s)	Broker down; no messages
250.9 s	Reconnected	`rc=0`; retained `offline` redelivered on subscribe
251.7 s	**Recovery confirmed**	`persons/telemetry` publishing resumes

Answer to Observations [Back to Question]

The Last Will was delivered by the broker almost immediately upon the broker's own shutdown (65.9 s, essentially the same instant as the `disconnected` event) rather than after any keepalive timeout — consistent with Mosquitto delivering queued wills as part of its own clean shutdown sequence, not something the subscriber had to wait out. Once the broker returned, telemetry publishing resumed cleanly within ∼1 s (`mosquitto_loop_start()`'s automatic reconnect handling working exactly as intended), confirming the board recovers without any manual intervention.
One genuine gap surfaced by this data: the `home/status` topic still reads the retained `offline` value at 250.9 s and does not appear to be republished as `online` anywhere in the captured window that follows, even though `persons/telemetry` data resumes normally moments later. A subscriber watching *only* the status topic (rather than inferring liveness from data still flowing) would therefore continue to see a stale `offline` value after an unplanned reconnect — worth fixing by explicitly republishing `"online"` from the MQTT `on_connect` callback, not only at initial process startup.

Experiment 3-5: Entry-to-Receipt Latency

Ten samples of end-to-end latency were collected — from a fresh detection timestamp appearing in `persons.json` to that same detection being received over MQTT by an external subscriber.

Table 15: Experiment 3-5 latency results (10 samples).

Statistic	Value
Mean	**1.008 s**
Standard deviation	0.020 s
Min	0.990 s
Max	1.033 s

Question What does this latency measurement actually capture? [Jump to Answer]

The assignment asks for mean and standard deviation of end-to-end latency across 10 samples, comparing timestamps.

Answer to Interpreting the result honestly [Back to Question]

The measured value is dominated by measurement resolution, not by the pipeline itself. `persons.json` timestamps (Section) only carry 1-second resolution, so every sample in this experiment necessarily lands close to a 1.0 s-plus-quantization-noise figure regardless of how fast the actual poll-and-publish pipeline

is — the true pipeline latency (detection poll interval plus MQTT publish/network round-trip) is very likely well under 100 ms, and is simply being masked by rounding a sub-second event to the nearest whole second before it ever reaches MQTT. The **very tight standard deviation** (0.020 s, i.e. roughly 2% of the mean) supports this reading: a genuinely variable network/processing latency would be expected to show more spread than a value that is mostly just "however far into the current second the real detection happened to land." The honest conclusion is that this experiment correctly demonstrates the system delivers detections within about a second end-to-end, but cannot resolve latency more precisely than that without a change to Section : having `person_detector.py` write a sub-second (float) epoch timestamp into `persons.json` instead of a whole-second ISO string would remove this quantization noise entirely and is recorded here as a concrete, specific future-work item rather than a vague "could be more precise."

Section Summary

Table 16: MQTT & Email deliverables completed in this section.

Item	**Status**
E-mail on detection (count + timestamp + temp + frame)	Complete
30-second debounce	Complete — Listing 8
MQTT publish to `home/persons/home/telemetry`	Complete, QoS 1
LWT configured before connect	Complete
Exp. 3-4 (broker outage, LWT reception)	Complete — Table 14
Exp. 3-5 (10-sample latency, mean/stdev)	Complete — Table 15